

Руководство по встроенным функциям Python для начинающих

Введение

Встроенные функции Python — это готовые инструменты, которые всегда доступны в языке без необходимости импорта дополнительных модулей. Они помогают решать повседневные задачи программирования быстро и эффективно.

Основные категории встроенных функций

1. Функции ввода и вывода

`print()`

Выводит текст или значения на экран.

```
python

print("Привет, мир!")
print("Возраст:", 25)
print("a =", 10, "b =", 20) # Можно выводить несколько значений
print("Python", "это", "круто!", sep=" - ") # Разделитель между значениями
```

`input()`

Считывает ввод пользователя как строку.

```
python

name = input("Введите ваше имя: ")
age = input("Введите ваш возраст: ")
age = int(age) # Преобразуем в число
print(f"Привет, {name}! Тебе {age} лет.")
```

2. Функции преобразования типов

`int()`

Преобразует значение в целое число.

```
python

int("123") # 123
int(45.7) # 45 (отбрасывает дробную часть)
int("1010", 2) # 10 (из двоичной системы)
```

float()

Преобразует значение в число с плавающей точкой.

```
python
```

```
float("3.14") # 3.14
float(42)     # 42.0
float("inf")  # inf (бесконечность)
```

str()

Преобразует значение в строку.

```
python
```

```
str(123)      # "123"
str(3.14)     # "3.14"
str([1, 2, 3]) # "[1, 2, 3]"
```

bool()

Преобразует значение в логический тип (True/False).

```
python
```

```
bool(1)       # True
bool(0)       # False
bool("")      # False (пустая строка)
bool("text")  # True
bool([])      # False (пустой список)
bool([1, 2])  # True
```

3. Математические функции

abs()

Возвращает абсолютное значение числа.

```
python
```

```
abs(-5) # 5
abs(3.14) # 3.14
abs(-2.7) # 2.7
```

round()

Округляет число до указанного количества знаков после запятой.

python

```
round(3.14159)    # 3
round(3.14159, 2) # 3.14
round(3.14159, 0) # 3.0
round(1234.5678, -2) # 1200.0 (округление до сотен)
```

max() и **min()**

Находят максимальное и минимальное значения.

python

```
max(1, 5, 3, 9, 2)    # 9
min(1, 5, 3, 9, 2)    # 1
max([10, 20, 5, 30])  # 30
min("apple", "banana", "cherry") # "apple" (по алфавиту)
```

sum()

Суммирует элементы итерируемого объекта.

python

```
sum([1, 2, 3, 4, 5])    # 15
sum((10, 20, 30))       # 60
sum([1, 2, 3], 100)     # 106 (начальное значение 100)
```

pow()

Возводит число в степень.

python

```
pow(2, 3)    # 8 (2 в степени 3)
pow(5, 2)    # 25
pow(2, 10, 3) # 1 (2^10 по модулю 3)
```

divmod()

Возвращает частное и остаток от деления.

```
python
```

```
divmod(17, 5) # (3, 2) - 17 = 5*3 + 2
```

```
divmod(20, 3) # (6, 2)
```

4. Функции для работы с последовательностями

len()

Возвращает длину объекта.

```
python
```

```
len("Python") # 6
```

```
len([1, 2, 3, 4]) # 4
```

```
len({"a": 1, "b": 2}) # 2
```

```
len(range(10)) # 10
```

range()

Создает последовательность чисел.

```
python
```

```
list(range(5)) # [0, 1, 2, 3, 4]
```

```
list(range(2, 8)) # [2, 3, 4, 5, 6, 7]
```

```
list(range(0, 10, 2)) # [0, 2, 4, 6, 8]
```

```
list(range(10, 0, -1)) # [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

enumerate()

Возвращает пары (индекс, значение) для элементов последовательности.

```
python
```

```
for i, value in enumerate(['a', 'b', 'c']):
```

```
    print(i, value)
```

```
# 0 a
```

```
# 1 b
```

```
# 2 c
```

```
list(enumerate("hello")) # [(0, 'h'), (1, 'e'), (2, 'l'), (3, 'l'), (4, 'o')]
```

zip()

Объединяет элементы нескольких последовательностей.

python

```
names = ["Анна", "Борис", "Вера"]
ages = [25, 30, 28]
for name, age in zip(names, ages):
    print(f"{name}: {age} лет")

list(zip([1, 2, 3], ['a', 'b', 'c'])) # [(1, 'a'), (2, 'b'), (3, 'c')]
```

reversed()

Возвращает итератор в обратном порядке.

python

```
list(reversed([1, 2, 3, 4])) # [4, 3, 2, 1]
list(reversed("hello"))     # ['o', 'l', 'l', 'e', 'h']
```

sorted()

Возвращает отсортированный список.

python

```
sorted([3, 1, 4, 1, 5])    # [1, 1, 3, 4, 5]
sorted("python")           # ['h', 'n', 'o', 'p', 't', 'y']
sorted([3, 1, 4], reverse=True) # [4, 3, 1]
sorted(["яблоко", "банан", "арбуз"], key=len) # ['банан', 'арбуз', 'яблоко']
```

5. Функции для работы с итерируемыми объектами

all()

Возвращает True, если все элементы истинны.

python

```
all([True, True, True])   # True
all([True, False, True])  # False
all([1, 2, 3])            # True
all([1, 0, 3])            # False (0 - ложь)
all([])                   # True (пустая последовательность)
```

any()

Возвращает True, если хотя бы один элемент истинен.

python

```
any([False, False, True]) # True
any([False, False, False]) # False
any([0, 0, 1]) # True
any([]) # False
```

map()

Применяет функцию к каждому элементу итерируемого объекта.

python

```
list(map(str, [1, 2, 3, 4])) # ['1', '2', '3', '4']
list(map(int, ['1', '2', '3'])) # [1, 2, 3]
list(map(len, ['hello', 'world'])) # [5, 5]

# С lambda-функцией
list(map(lambda x: x * 2, [1, 2, 3])) # [2, 4, 6]
```

filter()

Фильтрует элементы по условию.

python

```
list(filter(lambda x: x > 0, [-2, -1, 0, 1, 2])) # [1, 2]
list(filter(None, [0, 1, False, True, '', 'hello'])) # [1, True, 'hello']

def is_even(x):
    return x % 2 == 0

list(filter(is_even, [1, 2, 3, 4, 5, 6])) # [2, 4, 6]
```

6. Функции для работы с объектами

type()

Возвращает тип объекта.

python

```
type(42) # <class 'int'>
type("hello") # <class 'str'>
type([1, 2, 3]) # <class 'list'>
type(3.14) # <class 'float'>
```

isinstance()

Проверяет, является ли объект экземпляром определенного типа.

```
python

instance(42, int)    # True
instance("hello", str) # True
instance([1, 2], list) # True
instance(3.14, (int, float)) # True (проверка нескольких типов)
```

id()

Возвращает уникальный идентификатор объекта в памяти.

```
python

a = [1, 2, 3]
b = a
id(a) == id(b) # True (одинаковые объекты)

c = [1, 2, 3]
id(a) == id(c) # False (разные объекты)
```

dir()

Показывает доступные атрибуты и методы объекта.

```
python

dir("hello") # Список всех методов строки
dir([])      # Список всех методов списка
dir(42)      # Список всех методов числа
```

vars()

Возвращает словарь атрибутов объекта.

python

```
class Person:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
        self.age = 25
```

```
person = Person("Анна")
```

```
vars(person)  #{'name': 'Анна', 'age': 25}
```

7. Специальные функции

`ord()` и `chr()`

Работа с кодами символов.

python

```
ord('A')  # 65 (ASCII код)
```

```
ord('a')  # 1072 (Unicode код)
```

```
chr(65)   # 'A'
```

```
chr(1072) # 'a'
```

`bin()`, `oct()`, `hex()`

Преобразование чисел в различные системы счисления.

python

```
bin(10)   # '0b1010' (двоичная)
```

```
oct(10)   # '0o12' (восьмеричная)
```

```
hex(10)   # '0xa' (шестнадцатеричная)
```

`hash()`

Возвращает хеш-значение объекта.

python

```
hash("hello") # Хеш строки
```

```
hash(42)      # Хеш числа
```

```
hash((1, 2))  # Хеш кортежа
```

```
# hash([1, 2]) # Ошибка! Список не хешируется
```

Практические примеры

Пример 1: Обработка списка чисел

python

```
numbers = [1, -3, 5, -2, 8, 0, -7, 4]

print("Исходный список:", numbers)
print("Длина списка:", len(numbers))
print("Максимальное значение:", max(numbers))
print("Минимальное значение:", min(numbers))
print("Сумма:", sum(numbers))
print("Абсолютные значения:", list(map(abs, numbers)))
print("Положительные числа:", list(filter(lambda x: x > 0, numbers)))
print("Отсортированный список:", sorted(numbers))
```

Пример 2: Работа со строками

python

```
text = "Python Programming"

print("Текст:", text)
print("Длина:", len(text))
print("Все символы в верхнем регистре:", all(c.isupper() or c.isspace() for c in text))
print("Есть ли цифры:", any(c.isdigit() for c in text))
print("Коды символов:", list(map(ord, text[:6])))
print("Обратный порядок:", ".join(reversed(text)))
```

Пример 3: Пользовательский ввод и валидация

python

```
while True:
    user_input = input("Введите целое число: ")
    if user_input.isdigit() or (user_input.startswith('-') and user_input[1:].isdigit()):
        number = int(user_input)
        break
    else:
        print("Некорректный ввод. Попробуйте снова.")

print(f"Вы ввели: {number}")
print(f"Абсолютное значение: {abs(number)}")
print(f"Квадрат: {pow(number, 2)}")
```

Полезные советы

1. Используйте `help()` для получения справки о любой функции:

```
python  
  
help(len)  
help(sorted)
```

2. Комбинируйте функции для решения сложных задач:

```
python  
  
# Найти самое длинное слово  
words = ["python", "программирование", "код", "функция"]  
longest = max(words, key=len)
```

3. Проверяйте типы данных перед преобразованием:

```
python  
  
def safe_int(value):  
    if isinstance(value, str) and value.isdigit():  
        return int(value)  
    elif isinstance(value, (int, float)):  
        return int(value)  
    else:  
        return None
```

4. Используйте встроенные функции вместо циклов когда это возможно:

```
python  
  
# Вместо цикла  
total = 0  
for num in numbers:  
    total += num  
  
# Лучше использовать  
total = sum(numbers)
```

Заключение

Встроенные функции Python — это мощный инструмент, который делает код более читаемым, эффективным и pythonic. Изучение и применение этих функций поможет вам писать лучший код и решать задачи быстрее.

Помните: всегда есть встроенная функция, которая может упростить вашу задачу. Изучайте их, экспериментируйте и не бойтесь читать документацию!